

ReactTrendApplicationCompleteSetup andOperationsGuide

Version:2.0

LastUpdated:January10,2026

ProjectRepository: <https://github.com/Deepakking07/Trend-MiniProject02>

TableofContents

1. ExecutiveSummary
 2. ProjectOverview
 3. ArchitectureOverview
 4. Prerequisites
 5. InitialSetup
 6. InfrastructureDeployment
 7. ApplicationDeployment
 8. MonitoringSetup
 9. CI/CD Pipeline
 10. OperationalProcedures
 11. MonitoringandAlerting
 12. TroubleshootingGuide
 13. MaintenanceandOperations
 14. SecurityGuidelines
 15. CostOptimization
 16. DisasterRecovery
 17. Appendices
-

ExecutiveSummary

TheReactTrendApplication(Trendify)isaproduction-readye-commerceplatformbuilt withmodern DevOpspracticesanddeployedon AWScloudinfrastructureusingKubernetes orchestration.Thiscomprehensiveguideprovidesstep-by-stepinstructionsforsetup, deployment,monitoring,andongoingoperations.

KeyFeatures

- **Modern React Frontend:** E-commerce interface with responsive design and real-time product catalog
- **ContainerizedDeployment:** Docker containers orchestrated with Kubernetes onAWSEKS
- **Cloud Infrastructure:** AWS EKS cluster with auto-scaling capabilities across multiple AZs
- **CI/CD Automation:** Jenkins pipeline with automated testing, securityscanning, anddeployment

- **ComprehensiveMonitoring:**PrometheusandGrafana stackforfullobservability
- **HighAvailability:**Multi-AZdeploymentwithloadbalancingandauto-recovery

ProductionURLs

- **Application:** <http://a90433d6e6ec74243b4d2c732b930837-65a52193a1c515a6.elb.us-east-1.amazonaws.com/>
- **JenkinsCI/CD:** <http://3.236.24.216:8080/>
- **Grafana Monitoring:** <http://aa29945fe1c614be5ae0b521f91e2e87-1553642504.us-west-2.elb.amazonaws.com:3000>

ProjectOverview

BusinessContext

Trendify is an enterprise e-commerce platform designed to provide seamless shoppingexperienceswith:

- Productcatalogbrowsingwithadvancedsearch
- Shoppingcartfunctionalitywithpersistence
- Userauthenticationandauthorizationworkows Responsive
- design optimized for mobile and desktop devices Real-
- timeinventory updates

TechnicalStack

Component	Technology
Frontend	React18+withVitebuildsystem
WebServer	nginx1.29.1
Container Platform	Docker+Kubernetes(AWSEKS)
Infrastructure	AWS(VPC,EKS,EC2,ALB,RDS)
CI/CD	Jenkinswithdeclarativepipelines
Monitoring	Prometheus+Grafana+AlertManager
Container Registry	DockerHub
VersionControl	Git+GitHub
InfrastructureasCode	Terraform+YAML

Table1:TechnicalStackComponents

DeploymentModel

- **Environment:**Production on AWSEKS
- **Scaling:** Horizontal Pod Autoscaler (2-10pods)
- **LoadBalancing:**AWSApplication LoadBalancer(ALB) **Availability:**
- Multi-AZ deployment across us-west-2a and us-west-2b **Database:**
- RDS PostgreSQL (managed)
- **Storage:**S3forstaticassetsandbackups

Architecture Overview

High-Level System Architecture

The deployment architecture follows a modern cloud-native design:

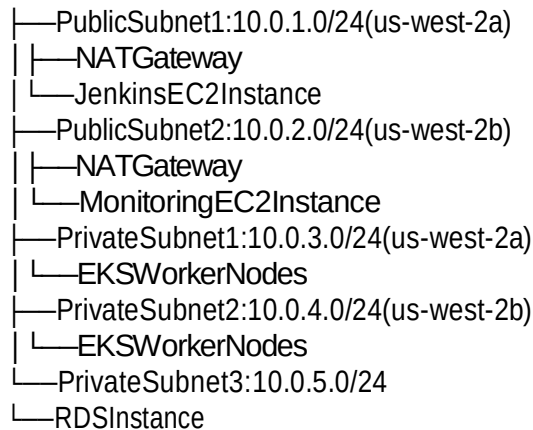
Figure 1: React Trend Application System Architecture

The architecture consists of:

1. **Developer Workstation** → Pushes code to GitHub
2. **GitHub Repository** → Triggers Jenkins webhook
3. **Jenkins CI/CD Server** → Builds, tests, and deploys
4. **AWS Cloud Infrastructure** → Hosts application and monitoring
5. **Docker Hub Registry** → Container image repository

VPC Network Architecture

VPC: 10.0.0.0/16



Component Architecture

1. Frontend Application Layer

- **Technology:** React 18+ with Vite
- **Purpose:** E-commerce user interface
- **Features:** Single Page Application (SPA), responsive design, product catalog, shopping cart, user authentication
- **Build System:** Vite for fast development and optimized production builds

2. Web Server Layer

- **Technology:** nginx 1.29.1
- **Configuration:** Port 3000, health checks at /health, metrics at /metrics
- **Purpose:** Static serving, routing, SSL/TLS termination
- **Load Balancing:** Integrated with Kubernetes Service

3. ContainerOrchestration

- Platform:** Kubernetes (AWS EKS)
- Components:** Deployments, Services, ConfigMaps, HPA, Ingress, NetworkPolicies •
- ContainerRuntime:** containerd
- CNI Plugin:** VPC CNI for AWS integration

4. InfrastructureLayer•

- Provider:** AWS
- Services:** EKS, VPC, ALB, EC2, IAM, S3, RDS, CloudWatch •
 - Networking:** VPC Endpoints for private connectivity

Prerequisites

RequiredToolsandVersions

Tool	Version	Purpose
AWS CLI	v2.x or later	AWS resource management
kubectl	v1.27+	Kubernetes cluster management
Terraform	v1.5+	Infrastructure as Code
Docker	v20.x+	Container management
Git	Latest	Version control
Node.js	v18+	Local development (optional)
Helm	v3.x+	Kubernetes package manager

Table 2: Required Tools and Versions

AWS Account Requirements

- **Permissions:** EC2, EKS, VPC, IAM, RDS, S3, CloudWatch full access
- **Credentials:** AWS Access Key and Secret Key configured
- **KeyPair:** EC2 key pair for SSH access to instances
- **Resource Limits:** Ensure sufficient VPC, EKS, and EC2 service quotas
- **IAM User:** User with Administrator Access for initial setup

External Account Requirements

- **Docker Hub:** Account with repository access for image storage
- **GitHub:** Repository access for source code and webhooks
- **Email:** For monitoring alerts and notifications

LocalEnvironmentSetup

Verify all tools are installed and configured correctly:

VerifyAWSCLI

aws sts get-caller-identity

```
root@ip-172-31-72-69:/home/ubuntu/MiniProject02# aws --version
aws-cli/2.32.16 Python/3.13.11 Linux/6.14.0-1015-aws exe/x86_64.ubuntu.24
root@ip-172-31-72-69:/home/ubuntu/MiniProject02# aws configure
AWS Access Key ID [*****XNQU]: AKIAZA4HCBKKO66RXNQU
AWS Secret Access Key [*****8SNQ]: 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
Default region name [us-east-1]: us-east-1
Default output format [None]:
root@ip-172-31-72-69:/home/ubuntu/MiniProject02# aws sts get-caller-identity
{
  "UserId": "AIDAZA4HCBKKGAJ4PYP6N",
  "Account": "620369152660",
  "Arn": "arn:aws:iam::620369152660:user/Cluster"
}
root@ip-172-31-72-69:/home/ubuntu/MiniProject02#
```

Verifykubectl

kubectl version --client

```
root@ip-172-31-72-69:/home/ubuntu/MiniProject02# kubectl version --client
Client Version: v1.34.3
Kustomize Version: v5.7.1
root@ip-172-31-72-69:/home/ubuntu/MiniProject02#
```

VerifyTerraform

terraform version

```
root@ip-172-31-72-69:/home/ubuntu/MiniProject02# terraform --version
Terraform v1.14.2
on linux_amd64
root@ip-172-31-72-69:/home/ubuntu/MiniProject02#
```

VerifyDocker

Docker version

```
GitCommit: de40ad0
root@ip-172-31-72-69:/home/ubuntu/MiniProject02# docker --version
Docker version 29.1.3, build f52814d
root@ip-172-31-72-69:/home/ubuntu/MiniProject02#
```

VerifyGit

git --version

```
root@ip-172-31-72-69:/home/ubuntu/workspace# git --version
git version 2.43.0
root@ip-172-31-72-69:/home/ubuntu/workspace#
```

InitialSetup

Step1:RepositoryCloning

Clonetheprojectrepositoryandnavigatetothe deploymentdirectory:

Clonethemainrepository

git clone <https://github.com/Deepakking07/Trend-MiniProject02.git>

cd Projects/MiniProject02

```
root@ip-172-31-72-69:/home/ubuntu/MiniProject02# git clone https://github.com/Deepakking07/Trend-MiniProject02.git
Cloning into 'Trend-MiniProject02'...
remote: Enumerating objects: 77, done.
remote: Counting objects: 100% (2/2), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 77 (delta 0), reused 0 (delta 0), pack-reused 75 (from 1)
Receiving objects: 100% (77/77), 8.58 MiB | 56.70 MiB/s, done.
Resolving deltas: 100% (1/1), done.
root@ip-172-31-72-69:/home/ubuntu/MiniProject02#
```

Verifyprojectstructure

ls-la

Expectedstructure:

- |—terraform/#InfrastructureasCode
- |—kubernetes/#K8smanifests
- |—src/#Reactapplicationsource
- |—Dockerle#Containerdenition
- |—Jenkinsle#CI/CDpipeline
- |—docker-compose.yml#Local development
- |—**README.md**

Step2: CongurationSetup

CongureTerraformvariableswithyourenvironmentdetails:

Copyterraformvariablestemplate

cpterraform/terraform.tfvars.exampleterraform/terraform.tfvars

Editcongurationle

nanoterraform/terraform.tfvars

RequiredCongurationValues:

```
project_name="trend-  
app"environment="producti  
on"region="us-west-2"  
key_name      =      "your-ec2-key-pair-  
name"availability_zones=["us-west-2a","us-  
west-2b"]
```

```
docker_username="your-dockerhub-  
username"jenkins_instance_type="t3.medium"  
monitoring_instance_type="t3.medium"  
eks_node_capacity=3  
eks_desired_capacity=3
```

Step3:DockerHubConguration

UpdateDockerHubreferencesacrosstheproject:

EditJenkinsle

```
sed-i's/your-username/your-dockerhub-username/g'Jenkinsle
```

EditKubernetesdeployment

```
sed-i's/your-username/your-dockerhub-username/g'kubernetes/deployment.yaml
```

Verifychanges

```
grep"your-dockerhub-username"Jenkinslekubernetes/deployment.yaml
```

Step4:EnvironmentVerication

ValidateyourAWSandKubernetessetup:

Check AWS connectivity and list EKS clusters

```
aws eks list-clusters --region us-west-2
```

VerifyTerraformcongruration

```
cd terraform  
terraform fmt #Format code  
terraform validate #Validatesyntax
```

Checkforanywarningsorerrors

```
echo"Congurationreadyfordeployment"
```

InfrastructureDeployment

Phase1:CoreInfrastructureDeployment

DeployAWSresourcesusingTerraform:

Navigatetoterraformdirectory

```
cdterraform
```

InitializeTerraform(downloadsproviders)

```
terraforminit
```

Reviewplannedinfrastructurechanges

```
terraformplan-var-le="terraform.tfvars"
```

Applyinfrastructurechanges(thistakes15-20minutes)

```
terraformapply-var-le="terraform.tfvars"
```

Saveoutputsforlateruse

```
terraform output > ../infrastructure-outputs.txt
```

ExpectedInfrastructureComponents:

- VPCwithpublic/privatesubnetsacross2AZs •
- EKSclusterwithmanagednodegroups
- Jenkins EC2 instance (t3.medium) for CI/CD
- MonitoringEC2instance(t3.medium)forPrometheus/Grafana •
- Securitygroupswithappropriateingress/egressrules
- IAMrolesandpoliciesforservice-to-servicecommunication •
- ApplicationLoadBalancerforexternaltrac
- RDSPostgreSQLinstanceforapplication database

EstimatedDeploymentTime:15-20minutes

Phase2:KubernetesConguration

CongurekubectltocommunicatewithyourEKScuster:

Updatekubecongwithcluster authentication

```
aws eks update-kubeconfig  
--region us-west-2  
--name trend-app-cluster
```

Verify cluster access

```
kubectl cluster-info
```

Check worker node status

```
kubectl get nodes
```

View node details

```
kubectl get nodes -o wide
```

Expected output:

NAME	STATUS	ROLES	AGE
------	--------	-------	-----

ip-10-0-3-xx.us-west-2.compute...	Ready	<none>	5m
-----------------------------------	-------	--------	----

ip-10-0-4-xx.us-west-2.compute...	Ready	<none>	5m
-----------------------------------	-------	--------	----

Phase3: Namespace Creation

Create Kubernetes namespaces for application isolation:

Create application namespace

```
kubectl apply -f kubernetes/namespace.yaml
```

Create monitoring namespace

`kubectl create namespace monitoring`

Verify namespace creation

`kubectl get namespaces`

Expected namespaces:

NAME STATUS AGE

default Active 10m

kube-system Active 10m

trend Active 1m

monitoring Active 1m

Infrastructure Verification Checklist

Before proceeding to application deployment:

- ☐ EKS cluster status is ACTIVE
- ☐ All worker nodes are in Ready state
- ☐ Jenkins instance is accessible via SSH []
- Monitoring instance is running
- ☐ Load balancer is provisioned and active
- ☐ Security groups allow necessary traffic
- ☐ RDS instance is available
- ☐ NAT gateways are operational
- ☐ VPC endpoints (if configured) are active

Application Deployment

Method 1: Automated Deployment Script

Use the provided deployment script for quick setup:

Makedeploymentscriptexecutable

chmod+xdeploy.sh

Deployallcomponents(namespace, congs,application)

./deploy.shall

Checkdeploymentstatus

./deploy.shstatus

Viewdeploymentlogs

./deploy.shlogs

Rollbackifneeded

./deploy.shrollback

Method2:ManualStep-by-StepDeployment

Step1:BuildandPushDockerImage

BuildtheReactapplicationandpushittoDocker Hub:

Navigatetoprojectroot

cdProjects/react-trend-deployment

BuildDockerimage

dockerbuild-tyour-dockerhub-username/trend-app:latest
--build-argBUILD_DATE=(date -u +' - - build - argVCS_REEF =(gitrev-parse--short HEAD).

Testimagelocally

dockerrun-p3000:3000your-dockerhub-username/trend-app:latest

Verify health endpoint responds

curl-I<http://localhost:3000/health>

Push to Docker Hub

docker login
docker push your-dockerhub-username/trend-app:latest

Step 2: Deploy Kubernetes Resources

Deploy all Kubernetes manifests in the correct order:

Create Config Maps for application configuration

kubectl apply -f kubernetes/congmap.yaml

Deploy the application

kubectl apply -f kubernetes/deployment.yaml

Expose the application

kubectl apply -f kubernetes/service.yaml

Configure auto-scaling

kubectl apply -f kubernetes/hpa.yaml

Optional: Configure ingress

kubectl apply -f kubernetes/ingress.yaml

Step 3: Verify Deployment

Verify that all resources were created successfully:

Check pod status

kubectl get pods -n trend
kubectl describe pods -n trend

Checkservicestatus

```
kubectlgetservices-ntrend  
kubectl describесervice trend-app-service-ntrend
```

Checkdeploymentstatus

```
kubectlgetdeployments-ntrend  
kubectl describе deployment trend-app-ntrend
```

Checkhorizontalpodautoscaler

```
kubectlgethpa-ntrend  
kubectl describе hpa trend-app-hpa-ntrend
```

Viewpodlogs

```
kubectllogs-ntrend deployment/trend-app--tail=50  
kubectllogs-ntrend deployment/trend-app-ffollowlogs
```

Executecommandsinpod

```
kubectlexec-it-ntrend deployment/trend-app--bin/sh  
kubectlexec -n trend deployment/trend-app--curllocalhost:3000/health
```

Step4:AccessApplication

Retrieve the load balancer URL and test the application:

GetloadbalancerURL

```
kubectlgetservice trend-app-service-ntrend-o  
jsonpath='{.status.loadBalancer.ingress[0].hostname}'
```

Testapplicationhealth

```
curl-Ihttp://<LOAD_BALANCER_URL>/health
```

Openinbrowser

```
openhttp://<LOAD_BALANCER_URL>
```

Testspecicendpoints

```
curlhttp://<LOAD_BALANCER_URL>/api/products  
curlhttp://<LOAD_BALANCER_URL>/api/health
```

Method3:CI/CDPipelineDeployment(Recommended)

UseJenkinsforautomated,repeatabledeployments.

JenkinsSetupandConguration

AccessJenkinsandcongurecredentials:

GetJenkinsIPfromTerraform output

```
JENKINS_IP=$(terraformoutput-rawjenkins_public_ip)
```

AccessJenkinsUI

open[http://\\${JENKINS_IP}:8080](http://${JENKINS_IP}:8080)

Getinitialadminpassword(viaSSH)

```
ssh-iyour-key.pemubuntu@${JENKINS_IP}  
sudocat/var/lib/jenkins/secrets/initialAdminPassword
```

CongureJenkinsCredentials

1. Navigateto**ManageJenkins**→**ManageCredentials**
2. AddDockerHubcredentials:
 - ID:** dockerhub-credentials
 - Username:**your-dockerhub-username ◦
 - Password:**your-dockerhub-token
3. AddAWScredentials:
 - ID:** aws-credentials
 - Access KeyID:** your-aws-access-key
 - Secret Access Key:**your-aws-secret-key

CreateJenkinsPipeline

1. NewItem→**Pipeline**
2. Congure:
 - Pipelinescript** from **SCM**
 - RepositoryURL:** YourGitHubrepository ◦
 - ScriptPath:**Jenkinsle
 - Branch:** main

PipelineExecution

Triggerthepipeline:

Manualtrigger

Navigate to Jenkins UI→Build Now

Webhooktrigger(onGitpush)

GitHub→RepositorySettings→Webhooks

URL: [http://\\$JENKINS_IP:8080/github-webhook/](http://$JENKINS_IP:8080/github-webhook/)

Scheduledtrigger

Jenkins Pipeline→ BuildTriggers → Poll SCM

Schedule:H*/4*** (Every4hours)

PipelineStagesinDetail

TheJenkinspipelineimplementsthefollowingstages:

1. CheckoutStage

ClonesthelatestcodefromGitHub:

```
stage('Checkout'){
  steps{
    gitbranch:'main',
    url:'https://github.com/your-username/trend-
app'sh'ls -la'
  }
}
```


2. Lint&SecurityScan

Performs code quality and security checks:

```
stage('Lint&SecurityScan'){
  parallel{
    stage('LintCode'){
      steps{
        sh'npm install'
        sh'npm run lint||true'
      }
    }
    stage('SecurityScan'){
      steps{
        sh'trivy fs.||true'
      }
    }
  }
}
```

3. BuildStages

Builds React application and Docker image:

```
stage('Build React App') {
  steps{
    sh'npm install'
    sh'npm run build'
  }
}

stage('Build Docker Image') {
  steps{
    script{docker.build("DOCKER_IMAGE :
{BUILD_NUMBER}")
  }
}
```

4. TestStages

Tests Docker image and application:

```
stage('Test Docker Image'){
  steps{
    sh""
    dockerrun--name test-${BUILD_NUMBER}
    -d DOCKER_IMAGE ${BUILD_NUMBER}
    sleep 5
    docker exec test-
    BUILD_NUMBER curl -f http://localhost:3000/health
    docker stop test-${BUILD_NUMBER}
    dockerrm test-${BUILD_NUMBER}
```

```
""  
}  
}
```

5. PushtoRegistry

Pushes image to Docker Hub:

```
stage('Push to DockerHub') {  
  steps{  
    script{  
      docker.withRegistry('https://registry.hub.docker.com',  
        'dockerhub-credentials'){  
        docker.image("${DOCKER_IMAGE} :${BUILD_NUMBER}").push('latest')  
      }  
    }  
  }  
}
```

6. Deploy to EKS

Deploy to Kubernetes cluster:

```
stage('Deploy to EKS'){  
  steps{  
    withAWS(credentials:'aws-credentials'){  
      sh""  
      aws eks update-kubeconfig --name trend-app-cluster --region us-west-2  
      kubectl set image deployment/trend-app  
      trend-app=${DOCKER_IMAGE} :${BUILD_NUMBER}  
      -n trend  
      kubectl rollout status deployment/trend-app -n trend ""  
    }  
  }  
}
```

Monitoring Setup

Phase 1: Monitoring Infrastructure

The monitoring stack is deployed as part of infrastructure:

- **Prometheus Server:** Scrapes metrics from Kubernetes and application
- **Grafana Dashboard:** Visualizes metrics and alerts
- **Alert Manager:** Routes alerts to notification channels
- **Node Exporter:** Collects system-level metrics

ImportDashboards

Import pre-built Kubernetes and application dashboards:

Dashboard ID to import:

- 15757: Kubernetes cluster overview**
- 8588: Node Exporter system metrics**

- 3662:Prometheusstats

- 1860: Node Exporter full metrics

ImportviaUI:

Dashboards→Import→Enterdashboard
ID→Load→Import

KeyMetricsMonitored

ApplicationMetrics

- **Availability:** `up{job="kubernetes-pods",pod=~"trend-app.*"}`
- **RequestRate:** `rate(nginx_http_requests_total[5m])`
- **ErrorRate:** `rate(nginx_http_requests_total{status=~"5.."}[5m])`
- **ResponseTime(p95):** `histogram_quantile(0.95, nginx_http_request_duration_seconds_bucket)`
- **CacheHitRatio:** `rate(nginx_cache_hits[5m])/(rate(nginx_cache_hits[5m]) + rate(nginx_cache_misses[5m]))`

InfrastructureMetrics

- **CPUUsage:** `rate(container_cpu_usage_seconds_total[5m])*100`
- **MemoryUsage:** `container_memory_usage_bytes/1024/ 1024`
- **NetworkI/O:** `rate(container_network_receive_bytes_total[5m])`
- **PodRestarts:** `increase(kube_pod_container_status_restarts_total[1h])`
- **DiskUsage:** `container_fs_usage_bytes/container_fs_limit_bytes*100`

KubernetesMetrics

- **PodStatus:** `kube_pod_status_phase`
- **Node Status:** `kube_node_status_condition`
- **DeploymentHealth:** `kube_deployment_status_replicas_available`
- **HPAStatus:** `kube_hpa_status_current_replicas`
- **PVCUsage:** `kubelet_volume_stats_used_bytes`

OperationalProcedures

DailyOperations

MorningHealthCheck

Performthesecheckseverymorning:

1. Checkapplicationaccessibility

```
curl-lhttp://<LOAD_BALANCER_URL>/health
```

2. Checkpodstatus

```
kubectlgetpods-ntrend  
kubectlgetpods-nmonitoring
```

3. Checkforrecenterrors

```
kubectllogs-n trenddeployment/trend-app--tail=100|grep-ierror
```

4. Checkresourceutilization

```
kubectltoppods-ntrend  
kubectl top nodes
```

5. CheckJenkinslastbuild

Navigate to Jenkins UI and verify last build status

6. CheckGrafanadashboardsfor anomalies

- ApplicationHealthStatus**
- PodCPUUsage**

- PodMemoryUsage

MorningHealthCheckChecklist:

- [] Application accessible via load balancer
- [] All pods in Running state
- [] No critical error spikes in logs
- [] Jenkins pipeline last run successful
- [] Resource utilization within normal ranges
- [] No pending pods
- [] Network connectivity healthy
- [] Database connections stable

IncidentResponse

If any check fails:

1. Assess the situation

```
kubectl describe pod <pod-name> -n trend  
kubectl logs <pod-name> -n trend
```

2. Check recent deployments

```
kubectl rollout history deployment/trend-app -n trend
```

3. If recent deployment caused issue

```
kubectl rollout undo deployment/trend-app -n trend
```

4. Scale up if resource-constrained

```
kubectl scale deployment trend-app --replicas=5 -n trend
```

5. Collect diagnostics for investigation

```
kubectl get events -n trend --sort-by='.lastTimestamp'
```

Weekly Operations

Perform these tasks weekly:

1. ResourceReview

```
kubectltop pods --ntrend --sort-by=cpu  
kubectltop pods --ntrend --sort-by=memory  
kubectl top nodes
```

2. HPAMetrics Review

```
kubectlggethpa --ntrend -o wide
```

3. CheckforPendingPods

```
kubectlgget pods --ntrend | grep Pending
```

4. UpdateBaseImages

```
docker pull node:18-alpine  
docker pull nginx:alpine
```

5. Review Grafana Dashboards

Check 7-day CPU and memory trends

Verify no resource spikes

Review error rate trends

6. Backup Configuration

```
kubectlgget all --ntrend -o yaml > backup-$(date +%Y%m%d).yaml
```

Monthly Operations

Infrastructure Review

1. AWS Cost Analysis

```
aws ce get-cost-and-usage  
--time-period Start=2026-01-01,End=2026-01-31  
--granularity MONTHLY  
--metrics BlendedCost
```


2. EKSClusterVersionCheck

```
aws eks describe-cluster  
--name trend-app-cluster  
--region us-west-2  
--query 'cluster.version'
```

3. SecurityGroupReview

```
aws sec2 describe-security-groups  
--filters "Name=group-name,Values=trend"
```

4. IAMPolicyAudit

```
aws iam list-role-policies --role-name trend-app-execution-role
```

5. BackupVerification

Verify Terraform state backups exist

Verify Kubernetes manifests backed up

Performance Optimization

1. AnalyzeHPAMetrics

```
kubectl get hpa trend-app-hpa -n trend -o yaml
```

2. Adjust resource limits if needed

```
kubectl set resources deployment trend-app  
-n trend  
--limits=cpu=500m,memory=1Gi  
--requests=cpu=250m,memory=512Mi
```

3. Review and optimize nginx config

```
kubectl get configmap nginx-config -n trend -o yaml
```

4. Analyzeslowqueries(ifapplicable)

Query application database for slow queries

MonitoringandAlerting

AlertRulesConguration

Criticalalertstriggeron:

groups:

- ♦ name:trend-app-critical
 - rules:
 - alert:ApplicationDown
 - expr: up{job="kubernetes-pods", pod=~"trend-app.*"} == 0
 - for:1m
 - labels:
 - severity:critical
 - annotations:
 - summary:"Trendapplicationisdown"alert:
 - HighErrorRate
 - expr:|
(rate/nginx_http_requests_total{status=~"5.."}[5m]) /
rate/nginx_http_requests_total[5m])) >0.05
 - for:5m
 - labels:
 - severity:critical
 - annotations:
 - summary:"Higherrorratedetected"

WarningAlerts

Warningalertsnotifyon:

- ♦ name:trend-app-warning
 - rules:
 - alert:HighCPUUsage
 - expr:|
(rate(container_cpu_usage_seconds_total{pod=~"trend-app.*"}[5m]) * 100) > 80
 - for:2m
 - labels:
 - severity:warning
 - annotations:
 - summary:"HighCPU
 - usagedetected"alert:HighMemoryUs
 - age
 - expr:|

```
(container_memory_usage_bytes{pod=~"trend-app.*"} /  
container_spec_memory_limit_bytes)>0.8  
for:2m  
labels:  
severity:warning  
annotations:  
summary:"Highmemoryusage detected"
```

TroubleshootingGuide

CommonIssuesand Solutions

Issue1: ApplicationNot Accessible

Symptoms:

- HTTPtimeoutwhenaccessingapplication
- Loadbalancerhealthchecks failing
- Serviceshowsno endpoints

Diagnosis:

Checkpointstatus

kubectlgetpods-ntrend

Checkservicedetails

kubectl describeservicetrend-app-service-ntrend

Checkendpoints

kubectlgetendpoints-ntrend

Testinternalconnectivity

kubectlexec -n trenddeployment/trend-app--curllocalhost:3000/health

Checklogs

kubectllogs-ntrenddeployment/trend-app--tail=50

Solutions:

Restartpodsifunhealthy

kubectlrolloutrestartdeployment/trend-app-ntrend

Verifyserviceselector

kubectlgetpods-ntrend--show-labelskubectleditservicetrend-app-service-ntrend

Checksecuritygroups

awssec2describe-security-groups--group-idssg-xxxxx

Verifyloadbalancer

kubectldescribesservice-trend-app-service-ntrend

Issue2: Pods Stuck in Pending State

Symptoms:

- Pods showPending status
- Unable to schedule on nodes
- Deploymenteventsshowsschedulingerrors

Diagnosis:

Describependingpods

kubectldescribepod-ntrend

Checknoderesources

kubectldescribenodes

Checknodestatus

kubectlgetnodes-owide

Checkforresourcerequests

kubectldescribedeploymenttrend-app-ntrend

Solutions:

Scale cluster if resource-constrained

```
aws eks update-nodegroup --cluster-name trend-app-cluster  
--nodegroup-name workers  
--scaling-config desiredSize=5
```

Remove taints if necessary

```
kubectl taint nodes --all key=value:NoSchedule-
```

Adjust pod resource requests

```
kubectl set resources deployment trend-app  
-n trend  
--requests=cpu=100m,memory=256Mi
```

Issue 3: Jenkins Pipeline Failures

Symptoms:

- Build fails during checkout
- Authentication errors with AWS or Docker
- Docker push fails

Diagnosis & Solutions:

Verify Jenkins credentials

Jenkins → Manage Jenkins → Manage Credentials

Check AWS CLI on Jenkins

```
ssh -i your-key.pem ubuntu@${JENKINS_IP}  
aws sts get-caller-identity
```

Verify Docker Hub access

```
docker login  
docker push test-image
```

CheckJenkinslogs

```
ssh-iyour-key.pem ubuntu@${JENKINS_IP}  
sudo tail -f /var/log/jenkins/jenkins.log
```

Issue4: Monitoring Data Missing

Symptoms:

- Grafana shows no data
- Prometheus targets marked down •
- No metrics visible in dashboards

Diagnosis:

Check Prometheus targets

```
kubectl port-forward -n monitoring svc/prometheus 9090:9090
```

Visit

<http://localhost:9090/targets> Check

k Prometheus logs

```
kubectl logs -n monitoring deployment/prometheus
```

Verify data source in Grafana

Grafana → Configuration → Data Sources Test

Prometheus connectivity

```
kubectl exec -n monitoring deployment/prometheus --  
nslookup kubernetes.default.svc.cluster.local
```

Solutions:

Restart monitoring components

```
kubectl rollout restart deployment/prometheus -n monitoring  
kubectl rollout restart deployment/grafana -n monitoring
```

Verifyscrapecongs

kubectlgetcongmap-nmonitoringprometheus-server-oyaml

Increasestorageiffull

kubectldeletepvcprometheus-storage-nmonitoring

SecurityGuidelines

AccessControl

Implementleast-privilege access:

CreateRBACfordevelopers

```
kubectlcreateroledeveloper
--verb=get,list,watch,create,update
--resource=pods,services,deployments
--namespace=trend
```

Bindroletousers

```
kubectlcreaterolebindingdeveloper-binding
--clusterrole=developer
--user=developer@example.com
--namespace=trend
```

NetworkSecurity

Congurenetworkpolicies:

```
apiVersion:networking.k8s.io/v1kind:
NetworkPolicy
metadata:
name:trend-app-policy
namespace:trend
spec:
podSelector:
matchLabels:
app:trend-app
policyTypes:
- Ingress
- Egress
ingress:
- from:
- namespaceSelector:
matchLabels:
```

```
name:ingress-nginx
egress:
- to:
- namespaceSelector:{}
```

ImageSecurity

Usesignedimagesandscanforvulnerabilities:

Buildwithsecurityscanning

```
dockerbuild-tyour-username/trend-app:latest.
trivyimageyour-username/trend-app:latest
```

Only deploy images with no critical vulnerabilities

```
dockerpushyour-username/trend-app:latest
```

SecretManagement

Managesensitivedatasecurely:

Createsecrets

```
kubectlcreatesecretgenericapp-secrets
--from-literal=db-password=your-password
-n trend
```

Referenceindeployments

env:

-name:DB_PASSWORD

valueFrom:

secretKeyRef:

name:app-secrets

key:db-password

CostOptimization

ResourceOptimization

Setappropriateresourcerequests/limits

```
kubectlsetresourcesdeploymenttrend-app  
-n trend  
--limits=cpu=500m,memory=1Gi  
--requests=cpu=250m,memory=512Mi
```

CongureHPAwithproperthresholds

**Current:70%CPUtrigger→scaleto10
podsmax**

Reviewandadjustbasedonactualusage

InfrastructureOptimization

**Usespotinstancesfornon-critical
workloads**

Editnodegroupcongrurationforspot instances

ReviewandoptimizeEBSvolumes

```
awscli2describe-volumes--ltersName=tag:Name,Values=trend-app
```

Consolidate resources during off-hours

Use Kubernetes Descheduler for bin-packing

Monitoring Cost

Review CloudWatch logs retention

`awslogs describe-log-groups`

Set appropriate retention periods

`awslogs put-retention-policy`

`--log-group-name/aws/eks/trend-app`

`--retention-in-days 7`

Disaster Recovery

Backup Strategy

1. Backup Terraform state

`cp terraform.tfstate terraform.tfstate.backup.$(date +%Y%m%d)`

2. Backup Kubernetes resources

`kubectl get all -n trend -o yaml > k8s-backup-$(date +%Y%m%d).yaml`

3. Backup application database

`kubectl exec -n trend deployment/trend-app --`

`mysqldump -u root -p database > backup-$(date +%Y%m%d).sql`

4. Store backups securely

`aws s3 cp terraform.tfstate.backup.* s3://trend-backup-bucket/`

`aws s3 cp k8s-backup-* s3://trend-backup-bucket/`

`aws s3 cp backup-*.sql s3://trend-backup-bucket/`

RecoveryProcedures

RestorefromTerraformbackup

```
cp terraform.tfstate.backup.20260110 terraform.tfstate
terraform apply -auto-approve
```

RestoreKubernetesresources

```
kubectl apply -f k8s-backup-20260110.yaml
```

Restoreapplicationdatabase

```
kubectl exec -i deployment/trend-app -- ntrend < backup-20260110.sql
```

Appendices

UsefulCommandsReference

KubernetesOperations:

- `kubectl get pods -n trend` - List pods
- `kubectl logs pod-name -n trend` - View logs
- `kubectl exec pod-name -n trend -- command` - Execute in pod
- `kubectl port-forward service/name 8080:3000 -n trend` - Port forward
- `kubectl top pods -n trend` - View resource usage
- `kubectl describe pod pod-name -n trend` - Detailed info
- `kubectl rollout restart deployment/name -n trend` - Restart deployment
- `kubectl scale deployment name --replicas=5 -n trend` - Scale pods

AWS CLI Operations:

- `aws eks describe-cluster --name trend-app-cluster` - Cluster info
- `aws eks list-clusters --region us-west-2` - List clusters
- `aws ec2 describe-instances` - List EC2 instances
- `aws s3 ls` - List S3 buckets
- `aws iam list-roles` - List IAM roles

Support and Escalation

For issues not covered in this guide:

1. **Check Kubernetes Events:** `kubectl get events -n trend --sort-by='.lastTimestamp'`
2. **Review Application Logs:** `kubectl logs -n trend deployment/trend-app-f`
3. **Check AWS CloudTrail:** Monitor AWS API calls
4. **Contact AWS Support:** For infrastructure issues
5. **Check Project Issues:** GitHub repository issues section

AdditionalResources

- ◆ [KubernetesDocumentationAW](#)
- ◆ [SEKSDocumentationTerrafor](#)
- ◆ [mAWSProviderPrometheusDoc](#)
- ◆ [umentationGrafanaDocumenta](#)
- ◆ [tionJenkinsDocumentation](#)
- ◆

PIPELINE SCREENSHOTS

The screenshot displays the Jenkins web interface for a build named 'Miniproject02'. The 'Console Output' tab is selected, showing the execution of a Jenkinsfile. The output indicates that the Jenkinsfile was obtained from a GitHub repository and that the pipeline is running on a Jenkins node. The main action shown is a Git checkout of the 'main' branch from a specific commit. The output includes the recommended Git tool, the resolution of the Git directory, the fetching of changes from the remote repository, and the checkout of the specified revision. The commit message is 'Update Jenkinsfile for Trend app pipeline'.

```
[Pipeline] f
[Pipeline] // stage
[Pipeline] withEnv (hide)
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout)
[Pipeline] git
The recommended git tool is: git
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/Miniproject02/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/Deepakking07/Trend-MiniProject02.git # timeout=10
Fetching upstream changes from https://github.com/Deepakking07/Trend-MiniProject02.git
> git --version # timeout=10
> git --version # 'git version 2.43.0'
> git fetch --tags --force --progress -- https://github.com/Deepakking07/Trend-MiniProject02.git +refs/heads/*:refs/remotes/origin/* #
timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 38a83dc3f187cefc7d8b555bb52bd2ecad9a4901 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 38a83dc3f187cefc7d8b555bb52bd2ecad9a4901 # timeout=10
> git branch -a -v --no-abbrev # timeout=10
> git branch -D main # timeout=10
> git checkout -b main 38a83dc3f187cefc7d8b555bb52bd2ecad9a4901 # timeout=10
Commit message: "Update Jenkinsfile for Trend app pipeline"
[Pipeline] script
```

```

> git branch -D main # timeout=10
> git checkout -b main 38a83dc3f187cefc7d8b555bb52bd2ecad9a4901 # timeout=10
Commit message: "Update Jenkinsfile for Trend app pipeline"
[Pipeline] script
[Pipeline] {
[Pipeline] sh
+ git rev-parse --short HEAD
[Pipeline] echo
Building image tag: 5-38a83dc
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build React App)
[Pipeline] script
[Pipeline] {
[Pipeline] sh
+ mkdir -p react-trend-deployment/dist
+ cd react-trend-deployment
+ [ -f package.json ]
+ ls -la dist/
total 8
drwxr-xr-x 2 jenkins jenkins 4096 Dec 14 20:05 .
drwxr-xr-x 3 jenkins jenkins 4096 Dec 14 20:05 ..
[Pipeline] }
[Pipeline] // script
[Pipeline] }

```

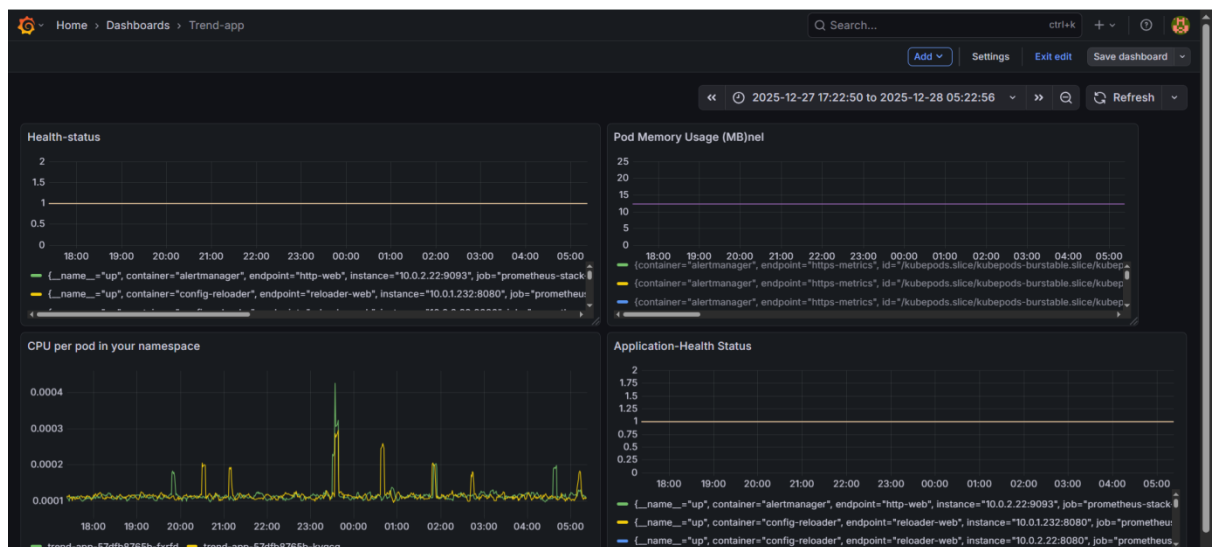
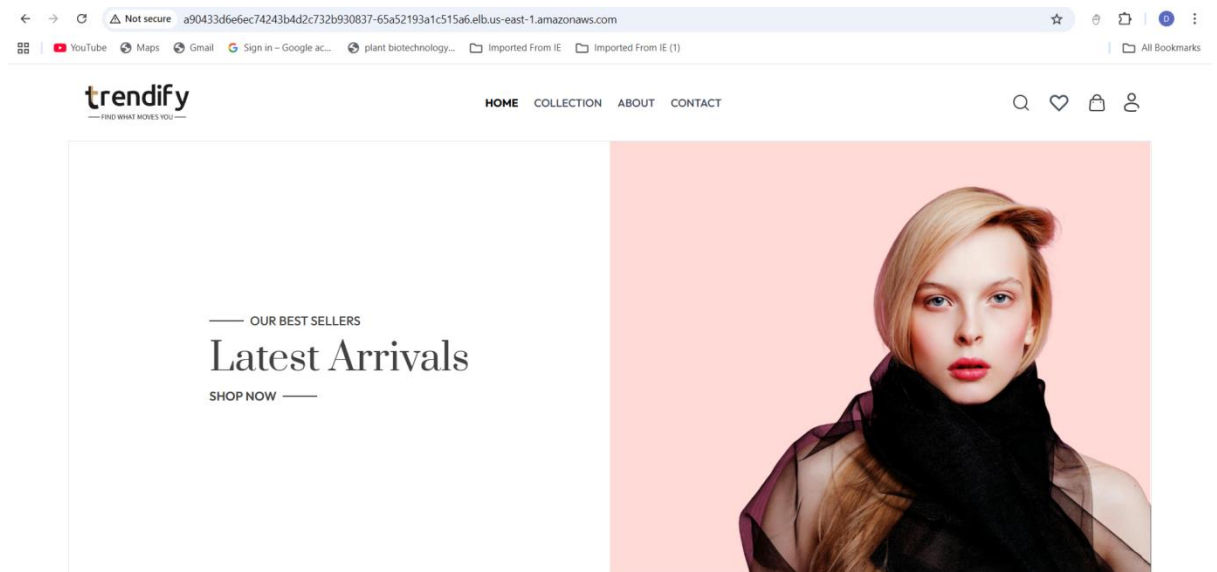
```

[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] sh
+ docker system prune -f
Deleted build cache objects:
fyc7vqcnfurh42e0yq49b3oaq
t73um96pg317jktlw8rsnlprn
szsq3p9agv1s0d5hcpisenv7u

Total reclaimed space: 9.421MB
[Pipeline] echo
Pipeline succeeded - Trend app deployed to EKS.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

FINAL OUTPUT



DocumentControl

- ◆ **Version:**2.0
- ◆ **LastUpdated:**January10,2026
- ◆ **Author:**DevOpsTeam
- ◆ **NextReviewDate:**April10,2026
- ◆ **ChangeLog:**
 - ov2.0(Jan 10,2026): Addedmonitoringsetupdetails,expandedtroubleshooting o
 - v1.0(Aug31,2025):Initialversion